Reporte Técnico de Actividades Práctico-Experimentales Nro. 007

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Rubén Sebastián Guartatanga Cano
Asignatura	Teoría de la programación
Ciclo	1
Unidad	2
Resultado de aprendizaje de la unidad	Aplica las estructuras de programación en la resolución de problemas básicos, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	007
Título de la Práctica	Aplicación de estructuras condicionales en la resolución de problemas **Fase 2 - Codificación y validación de programa**
Nombre del Docente	José Oswaldo Guamán Quinche
Fecha	Jueves 12 de noviembre del 2026 al Viernes 13 de noviembre del 2026
Horario	Jueves, 10h30 – 11h30 Viernes, 09h30 - 11h30
Lugar	Aula física asignada al paralelo.
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

- Comprender y aplicar las estructuras condicionales simples, dobles y múltiples en la resolución de problemas.
- Analizar, diseñar y codificar un algoritmo que utilice sentencias de decisión para analizar y clasificar información.
- Validar el funcionamiento del programa mediante la ejecución práctica.

3. Materiales, Reactivos, Equipos y Herramientas

- IDE de programación: Visual Studio Code.
- Lenguaje de programación: Java.
- Computador personal con sistema operativo Windows.
- Material de apoyo en el Entorno Virtual de aprendizaje (EVA).
- Editor de texto Word para la documentación de la actividad.
- Conexión a internet estable.

4. Procedimiento / Metodología Ejecutada

a) Se analizó los datos de entrada, el proceso y salida del problema presentado en el APE 6 del EVA. Luego, se elaboró una tabla con estos datos ya reconocidos para facilitar la comprensión del problema.
b) Con el análisis, se desarrolló un pseudocódigo que permita comprender los pasos a seguir en la elaboración y ejecución del programa.
c) Se diseñó el código fuente en el lenguaje de programación Java, guiándonos con el pseudocódigo previamente realizado.
d) Se probó la funcionalidad del programa en una prueba de escritorio con las siguientes calificaciones: 9.00 (evaluación individual), 7.50 (trabajo grupal) y 8.00 (evaluación final).

5. Resultados

1) Análisis

En la siguiente tabla se presenta el análisis del problema (Datos de entrada, proceso y salida).

Tabla 1

Análisis del problema

Datos de entrada	Nota individual.
	Nota grupal.
	Nota de evaluación final
Proceso	Validar que las calificaciones no excedan el rango (0 a 10).
	Calcular la nota final teniendo en cuenta sus respectivos porcentajes.
	Determinar el nivel de desempeño del estudiante en base a sus calificaciones.
	Comprobar si aprobó o reprobó.
	Calificaciones ingresadas.
Datos de salida	Mostrar la nota final.
	Mostrar el nivel de desempeño del estudiante.
	Mostrar el estado del estudiante (aprobado o reprobado).
	Mensaje adicional según la nota final.

2) Diseño del algoritmo (pseudocódigo)

Inicio
Definir notaIndividual, notaGrupal, notaEvaluacion, notaFinal Como Real
Definir desempeno, estado Como Cadena
Escribir “Ingresar la nota individual”
Leer notaIndividual
Escribir “Ingresar la nota grupal”
Leer notaGrupal
Escribir “Ingresar la nota de la evaluación final”
Leer notaEvaluacion

```
Si (notaIndividual >= 0) Y (notaIndividual <= 10) Y (notaGrupal >= 0) Y (notaGrupal <= 10) Y (notaEvaluacion >= 0) Y (notaEvaluacion <= 10) Entonces
    notaFinal <- (notaIndividual * 0.30) + (notaGrupal * 0.30) + (notaEvaluacion * 0.40)
    Si (notaFinal >= 9) Entonces
        desempeno <- “Excelente”
    Sino
        Si (notaFinal >= 8) Entonces
                desempeno <- “Muy bueno”
        Sino
                Si (notafinal >= 7) Entonces
                        desempeno <- “Bueno”
                Sino
                        Si (notaFinal >= 5) Entonces
                                desempeno <- “Regular”
                        Sino
                                desempeno <- “Deficiente”
                        FinSi
                FinSi
        FinSi
    FinSi
    Si (notaFinal >= 7) Entonces
        estado <- “Aprobado”
    Sino
        estado <- “Reprobado”
        Escribir “El estudiante requiere refuerzo academico”
    FinSi
    Si (notaFinal >= 9) Entonces
        Escribir “¡Felicitaciones! Rendimiento sobresaliente"
    FinSi
    Escribir "Nota individual: ", notaIndividual
    Escribir "Nota grupal: ", notaGrupal
    Escribir "Nota evaluacion final: ", notaEvaluacion
    Escribir "Promedio ponderado: ", notaFinal
    Escribir "Nivel de desempeno: ", desempeno
    Escribir "Estado final: ", estado
Sino
    Escribir “Los datos ingresados no son válidos.”
FinSi
```

3) Código fuente

```
import java.util.Scanner;
import java.util.Locale;
```

```
public class APE7 {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);
      sc.useLocale(Locale.US);
//Definición de variables.
      double notaIndividual, notaGrupal, notaEvaluacion, notaFinal;
      String desempeno, estado;
//Datos de entrada: nota individual, nota grupal y nota de la evaluación final.
      System.out.println("Ingrese la nota individual:");
      notaIndividual = sc.nextDouble();

      System.out.println("Ingrese la nota grupal:");
      notaGrupal = sc.nextDouble();

      System.out.println("Ingrese la nota de la evaluacion final:");
      notaEvaluacion = sc.nextDouble();
//Validación de los datos ingresados: Mayores o iguales que
//cero y menores o iguales que 10 para que no excedan el límite
      if (notaIndividual >= 0 && notaIndividual <= 10 &&
        notaGrupal >= 0 && notaGrupal <= 10 &&
        notaEvaluacion >= 0 && notaEvaluacion <= 10) {
//Proceso: Cálculo de la nota final.
        notaFinal = (notaIndividual * 0.30) +
                (notaGrupal * 0.30) +
                (notaEvaluacion * 0.40);
//Proceso: Definición del desempeño basado en la nota final.
        if (notaFinal >= 9) {
          desempeno = "Excelente";
        } else {
          if (notaFinal >= 8) {
            desempeno = "Muy bueno";
          } else {
            if (notaFinal >= 7) {
              desempeno = "Bueno";
            } else {
              if (notaFinal >= 5) {
                desempeno = "Regular";
              } else {
                desempeno = "Deficiente";
              }
            }
```

```
            }
        }
//Estado del estudiante: Si la nota final es mayor o igual a 7 el estudiante está aprobado.
//Si la nota final es menor a 7 el estudiante está reprobado y necesita un refuerzo académico.
        if (notaFinal >= 7) {
            estado = "Aprobado";
        } else {
            estado = "Reprobado";
            System.out.println("El estudiante requiere refuerzo academico");
        }
//Si la nota final del estudiante el mayor o igual a 9 su rendimiento es sobresaliente.
        if (notaFinal >= 9) {
            System.out.println("¡Felicitaciones! Rendimiento sobresaliente");
        }
//Datos de salida: Se muestran: las notas ingresadas, la nota final, el desempeño del estudiante
//y el estado del mismo (aprobado o reprobado).
        System.out.println("\nRESULTADOS");
        System.out.printf("Nota individual: %.2f%n", notaIndividual);
        System.out.printf("Nota grupal: %.2f%n", notaGrupal);
        System.out.printf("Nota evaluacion final: %.2f%n", notaEvaluacion);
        System.out.printf("Promedio ponderado: %.2f%n", notaFinal);
        System.out.println("Nivel de desempeno: " + desempeno);
        System.out.println("Estado final: " + estado);
//Mensaje de error si los valores ingresados no están dentro del límite.
    } else {
        System.out.println("Los datos ingresados no son validos.");
    }

    sc.close();
  }
}
```

4) Prueba de escritorio

En la siguiente tabla se presenta una prueba de escritorio a partir de los siguientes datos: 9.00 (evaluación individual), 7.50 (trabajo grupal) y 8.00 (evaluación final).

Tabla 2

Prueba de escritorio.

Variable / Proceso	Resultado
notaIndividual	9.00
notaGrupal	7.50
notaEvaluacion	8.00
Validación de notas	Verdadero

notaFinal	8.15
notaFinal >= 9	Falso
notaFinal >= 8	Verdadero
desempeno	Muy bueno
notaFinal >= 7	Verdadero
estado	Aprobado
Mensaje de refuerzo	No se muestra
Mensaje de felicitación	No se muestra

6. Preguntas de Control

¿Qué es una condición y cuál es su función dentro de una estructura condicional en programación?

Una condición se define como una expresión lógica que permite verificar si una situación es verdadera o falsa. Su función en el programa es que este mismo ejecute cierta acción dependiendo si se cumple o no la condición.

¿Qué diferencia existe entre una estructura condicional simple, doble y múltiple?

Condicional simple (if): se usa para ejecutar acciones únicamente si la condición es verdadera.

Condicional doble (if - else): Sirve para ejecutar dos alternativas dependiendo si la condición es falsa o verdadera.

Condicional múltiple: Permite evaluar varias condiciones y seleccionar la instrucción a ejecutar dependiendo del caso que se cumpla.

¿Qué es una estructura condicional anidada y en qué casos resulta útil emplearla dentro de un programa?

Consiste en colocar una estructura if – else dentro de otra estructura condicional. Se usa cuando un programa necesita evaluar consecutivamente condiciones para tomar decisiones precisas [1].

7. Conclusiones

- A través de la resolución del problema planteado en el APE 6, se ha comprendido y aplicado de manera efectiva las estructuras condicionales simples, dobles y múltiples en el código fuente.
- Se analizó, diseñó y codificó de manera satisfactoria un algoritmo que utiliza sentencias de decisión para el análisis y clasificación con el objetivo de satisfacer las necesidades del problema presentado.
- A través de la ejecución práctica se validó exitosamente la ejecución del programa, comprobando su correcto funcionamiento y otorgando una resolución efectiva al problema

8. Recomendaciones

Verificar la estructura de las condicionales para evitar problemas en la ejecución del programa.

9. Bibliografía / Referencias

[1] W. Maalej, “On Fun for Teaching Large Programming Courses,” in *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*, Rio de Janeiro, Brazil, 2026, pp. 256–266, doi: 10.1145/3786580.3786988.

10. Anexos

https://github.com/Thelsi/Teoria_Programacion